

# Procesamiento del Lenguaje Natural

## Semántica vectorial

---

Jorge Ortiz Fuentes

18 de marzo de 2025

- ¿Cómo recuperan documentos relevantes los motores de búsqueda como Duckduckgo o Google a partir de una consulta?

Estos problemas se estudian en el campo de *Information Retrieval*: la ciencia de la búsqueda de información en colecciones de documentos.

¿Cómo puede una empresa procesar los reclamos que los usuarios dejan en sus portales web?

Estos problemas se estudian en los siguientes campos:

- *Text Mining*: extracción automática de conocimiento a partir de texto.

¡Ambos están estrechamente relacionados con el NLP! (las fronteras entre estos campos no son claras)

- La tokenización es la tarea de dividir una oración o documento en piezas llamadas *tokens*.
- Se pueden emplear transformaciones adicionales como la eliminación de caracteres especiales (puntuación), convertir a minúsculas, etc. [Manning et al., 2008].

## Ejemplo

Entrada:

Me gustan los perros y los gatos.

Tokens:

[me] [gustan] [los] [perros] [y] [los] [gatos]

- Un *type* es una clase de *token* que contiene una única secuencia de caracteres.
- Se obtienen identificando tokens únicos dentro del documento.

## Ejemplo

Entrada:

Me gustan los perros y los gatos.

Types:

[me] [gustan] [los] [perros] [y] [gatos]

Observación: Note que el token *los* aparece una sola vez en la lista de types a pesar de repetirse en la oración.

- Un *term* es un *type* normalizado.
- La normalización es el proceso de crear clases de equivalencia de diferentes *types*. Esto se aclarará en las siguientes diapositivas.
- El vocabulario  $V$ , es el conjunto de términos (tokens únicos normalizados) dentro de una colección de documentos o corpus  $D$ .

## Extracción de vocabulario (2/2)

- Para reducir el tamaño del vocabulario y eliminar términos que no proporcionan mucha información, se eliminan los términos que ocurren con alta frecuencia en el corpus.
- Estos términos se llaman *stopwords* e incluyen artículos, pronombres, preposiciones y conjunciones.  
Ejemplo: [un, una ,la, el, los, a, ante, bajo, con].<sup>1</sup>

### Ejemplo de eliminación de Stopwords

Entrada: No me gusta la pizza

Output: [gusta] [pizza]

---

<sup>1</sup>Conceptos relacionados: palabras funcionales, palabras de clase cerrada.

Un proceso de normalización de términos en el que los términos se transforman a su raíz.

| Regla     | Ejemplo           |
|-----------|-------------------|
| SSES → SS | caresses → caress |
| IES → I   | ponies → poni     |
| SS → SS   | caress → caress   |
| S →       | cats → cat        |

**Cuadro 1:** Ejemplo de reglas de stemming del Algoritmo de Porter

### Ejemplo

Entrada:

I like human languages and programming languages

Después de stemming:

I like human languag and programm languag<sup>2</sup>

El vocabulario del documento *d* después de eliminar las stopwords y realizar stemming:

| termId | value   |
|--------|---------|
| t1     | human   |
| t2     | languag |
| t3     | program |

---

<sup>2</sup>[http://9ol.es/porter\\_js\\_demo.html](http://9ol.es/porter_js_demo.html)

### Ventajas del stemming:

- Reduce el tamaño del vocabulario
- Agrupa palabras relacionadas morfológicamente
- Mejora el recall en sistemas de recuperación de información

### Desventajas:

- Puede generar stems que no son palabras reales
- Puede agrupar palabras con significados diferentes
- No considera el contexto de la palabra

- Otra estrategia de normalización de términos.
- También transforma las palabras a sus raíces.
- Realiza un análisis morfológico utilizando diccionarios de referencia (tablas de búsqueda) para crear clases de equivalencia entre *types*.
- Por ejemplo, para el token *studies*, una regla de stemming devolvería el término *studi*, mientras que a través de la lematización obtendríamos el término *study*<sup>3</sup>.

---

<sup>3</sup><https://blog.bitext.com/what-is-the-difference-between-stemming-and-lemmatization/>

## Ley de Zipf (1/4)

- Para construir un vocabulario eficiente, necesitamos entender cómo se distribuyen las palabras en los textos.
- La ley de Zipf, propuesta por *George Kingsley Zipf* en [Zipf, 1935], es una ley empírica que describe este fenómeno.
- Nos ayuda a:
  - Identificar palabras comunes (potenciales stopwords)
  - Optimizar el almacenamiento del vocabulario
  - Entender patrones lingüísticos naturales

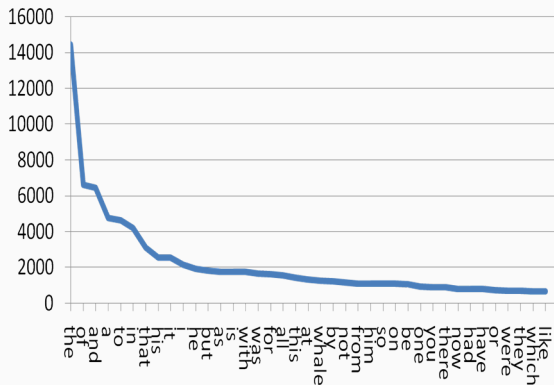
## Ley de Zipf (2/4)

- Establece que la frecuencia  $f$  de un término en un corpus es inversamente proporcional a su rango  $r$  en una tabla de frecuencias ordenada:

$$f = \frac{cf}{r^\beta} \quad (1)$$

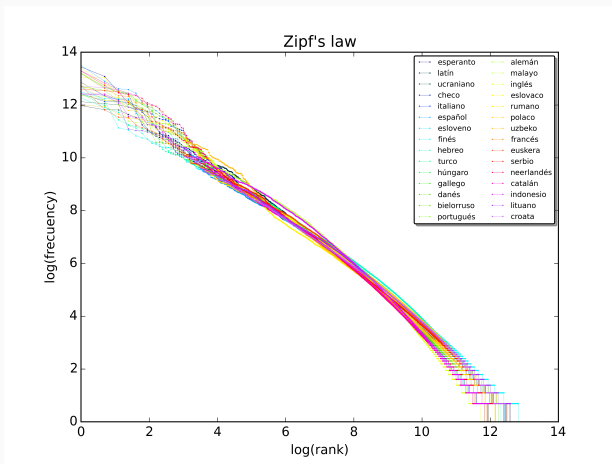
- Donde:
  - $cf$  es una constante dependiente de la colección
  - $\beta > 0$  es un factor de decaimiento
  - Si  $\beta = 1$ , sigue exactamente la ley de Zipf
- La ley se relaciona con el principio del mínimo esfuerzo: tendemos a usar un conjunto reducido de palabras con alta frecuencia.

## Ley de Zipf (3/4)



- Listar las palabras más frecuentes de un corpus se puede utilizar para construir una lista de *stopwords*.

## Ley de Zipf (4/4)



Si graficamos un gráfico *log-log*, obtenemos una línea recta con pendiente  $-\beta$ .

## Representación Vectorial de Texto (1/2)

- El procesamiento de texto requiere convertir las palabras en números para poder realizar cálculos matemáticos.
- La representación vectorial permite:
  - Calcular similitudes entre documentos
  - Agrupar documentos similares
  - Clasificar documentos
  - Encontrar documentos relevantes para una consulta

## Representación Vectorial de Texto (2/2)

- Una forma básica consiste en *representar* los documentos como vectores de términos, donde cada término es una dimensión vectorial [Salton et al., 1975].
- Documentos con diferentes palabras y longitudes residirán en el mismo espacio vectorial.
- A continuación veremos una forma clásica para representar palabras como vectores llamada *Bag of Words* y una variación para ponderar las representaciones llamada *TF-IDF*.

## Bag of Words (1/3)

- La representación más simple y común de texto como vector es el modelo *Bag of Words* (BoW).
- En BoW, un documento se representa como una “bolsa” (multiconjunto) de sus palabras.
- Características principales:

| Lo que ignora         | Lo que considera               |
|-----------------------|--------------------------------|
| Orden de las palabras | Frecuencia de cada palabra     |
| Estructura gramatical | Presencia/ausencia de términos |
| Sintaxis              | Distribución estadística       |

## Bag of Words (2/3)

- El valor de cada dimensión es un peso que representa la relevancia del término  $t_i$  en el documento  $d$ :

$$d_j \rightarrow \vec{d_j} = (w(t_1, d_j), \dots, w(t_{|V|}, d_j)) \quad (2)$$

### Ejemplo

Texto original: El gato come pescado y el perro come carne

Representación BoW:

| Término | Frecuencia |
|---------|------------|
| el      | 2          |
| gato    | 1          |
| come    | 2          |
| pescado | 1          |
| y       | 1          |
| perro   | 1          |
| carne   | 1          |

## Bag of Words (3/3)

| Ventajas                                                                                                  | Desventajas                                                                                                   |
|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Simple de implementar<br>Computacionalmente eficiente<br>Funciona para tareas básicas<br>Fácil de escalar | Pierde información contextual<br>No captura la semántica<br>No maneja bien negaciones<br>Alta dimensionalidad |

- A pesar de sus limitaciones, BoW es una representación fundamental que sirve como base para modelos más sofisticados.
- Es especialmente útil en tareas como:
  - Clasificación de documentos
  - Recuperación de información
  - Análisis de sentimientos básico

## Term Frequency - Inverse Document Frequency (TF-IDF)

- TF-IDF es una medida estadística que evalúa qué tan relevante es una palabra para un documento en una colección
- Combina dos métricas:
  - **Term Frequency (TF):** Frecuencia de la palabra en el documento
  - **Inverse Document Frequency (IDF):** Qué tan única es la palabra en toda la colección
- Intuición:
  - Si una palabra aparece mucho en un documento, probablemente sea importante (TF alto)
  - Si una palabra aparece en pocos documentos, probablemente sea más distintiva (IDF alto)

## Term Frequency (TF)

- Term Frequency mide la frecuencia de un término en un documento
- Existen diferentes formas de calcular TF:

| Variante               | Fórmula                                                                                        |
|------------------------|------------------------------------------------------------------------------------------------|
| Frecuencia Bruta       | $tf_{i,j} = f_{i,j}$                                                                           |
| Frecuencia Binaria     | $tf_{i,j} = \begin{cases} 1 & \text{si } f_{i,j} > 0 \\ 0 & \text{caso contrario} \end{cases}$ |
| Frecuencia Logarítmica | $tf_{i,j} = 1 + \log(f_{i,j})$                                                                 |
| Frecuencia Normalizada | $tf_{i,j} = \frac{f_{i,j}}{\max_k f_{k,j}}$                                                    |

Donde  $f_{i,j}$  es la frecuencia del término  $i$  en el documento  $j$

## Inverse Document Frequency (IDF)

- IDF mide qué tan común o raro es un término en toda la colección
- Penaliza términos que aparecen en muchos documentos
- Fórmula básica:

$$idf_i = \log \left( \frac{N}{n_i} \right)$$

- $N$  = número total de documentos
- $n_i$  = número de documentos que contienen el término  $i$

| Término      | Docs con término | Total docs | IDF  |
|--------------|------------------|------------|------|
| el           | 98               | 100        | 0.02 |
| algoritmo    | 5                | 100        | 3.00 |
| pelotillehue | 1                | 100        | 4.61 |

- Palabras comunes como “el” tienen IDF bajo y palabras raras como “pelotillehue” tienen IDF alto

## Cálculo Final de TF-IDF (1/2)

- El peso final TF-IDF se calcula multiplicando  $TF \times IDF$ :

$$w_{i,j} = tf_{i,j} \times idf_i$$

### Ejemplo

Documento: “El gato come pescado. El gato duerme”

| Término | TF | IDF  | TF-IDF | Interpretación |
|---------|----|------|--------|----------------|
| el      | 2  | 0.02 | 0.04   | Poco relevante |
| gato    | 2  | 2.30 | 4.60   | Muy relevante  |
| come    | 1  | 2.70 | 2.70   | Relevante      |
| pescado | 1  | 3.00 | 3.00   | Relevante      |
| duerme  | 1  | 2.70 | 2.70   | Relevante      |

- Las palabras con alto TF-IDF son las más características del documento
- Este peso se usa para construir los vectores de documentos
- Propiedades importantes del TF-IDF:
  - Asigna mayor peso a términos frecuentes en pocos documentos
  - Penaliza términos que aparecen en muchos documentos
  - Es independiente del tamaño del documento
  - Facilita la comparación entre documentos

## Similitud entre Vectores (1/2)

- Representar consultas y documentos como vectores permite calcular su similitud.
- Un enfoque sería usar la distancia euclidiana.
- El enfoque común es calcular el coseno del ángulo entre los dos vectores.
- Si ambos documentos son iguales:
  - El ángulo sería 0
  - Su coseno sería 1
- Si los documentos son ortogonales:
  - El coseno es 0

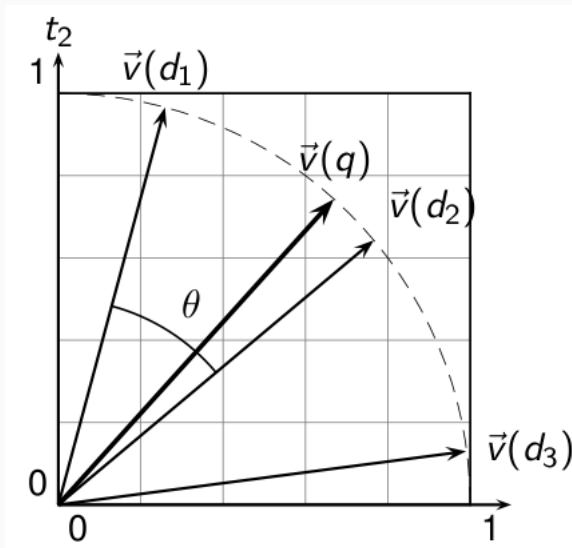
## Similitud entre Vectores (2/2)

- La similitud del coseno se calcula de la siguiente manera:

$$\cos(\vec{d}_1, \vec{d}_2) = \frac{\vec{d}_1 \cdot \vec{d}_2}{|\vec{d}_1| \times |\vec{d}_2|} = \frac{\sum_{i=1}^{|V|} (w(t_i, d_1) \times w(t_i, d_2))}{\sqrt{\sum_{i=1}^{|V|} w(t_i, d_1)^2} \times \sqrt{\sum_{i=1}^{|V|} w(t_i, d_2)^2}}$$

- Esto se llama erróneamente *distancia del coseno*. En realidad, es una métrica de similitud.
- Tenga en cuenta que la similitud del coseno normaliza los vectores por su norma euclidiana  $\|\vec{d}\|_2$ .

## Similitud del Coseno



**Figura 1:** Similitud del Coseno.

## Ejercicio

- Supongamos que tenemos 3 documentos formados a partir de las siguientes secuencias de términos:

$$d_1 \rightarrow t_4 t_3 t_1 t_4$$

$$d_2 \rightarrow t_5 t_4 t_2 t_3 t_5$$

$$d_3 \rightarrow t_2 t_1 t_4 t_4$$

- Construya una matriz de términos-documentos de  $5 \times 3$  dimensiones usando pesos *tf-idf* simples (sin normalización).
- Le recomendamos que primero construya una lista con el número de documentos en los que aparece cada término (útil para calcular las puntuaciones *idf*)
- Luego calcule las puntuaciones *idf* de cada término.
- Llene las celdas de la matriz con los valores *tf-idf*.
- ¿Cuál es el documento más cercano a  $d_1$ ?

**Cuadro 2:** Matriz tf-idf

|    | d1    | d2    | d3    |
|----|-------|-------|-------|
| t1 | 0.176 | 0.000 | 0.176 |
| t2 | 0.000 | 0.176 | 0.176 |
| t3 | 0.176 | 0.176 | 0.000 |
| t4 | 0.000 | 0.000 | 0.000 |
| t5 | 0.000 | 0.954 | 0.000 |

## N-gramas (1/2)

- Los n-gramas son secuencias contiguas de n elementos (palabras o caracteres).
- Tipos comunes:
  - Unigramas ( $n=1$ ): palabras individuales
  - Bigramas ( $n=2$ ): pares de palabras consecutivas
  - Trigramas ( $n=3$ ): tríos de palabras consecutivas
- Aplicaciones en BoW y TF-IDF:
  - Se pueden usar como términos en vez de palabras individuales
  - Permiten capturar frases y expresiones completas
  - Mejoran la precisión en búsquedas de frases exactas
- Desventajas:
  - Mayor dispersión de los vectores
  - Mayor costo computacional

### Ejemplo

``El gato negro duerme''`

- Unigramas: [el] [gato] [negro] [duerme]
- Bigramas: [el gato] [gato negro] [negro duerme]
- Trigramas: [el gato negro] [gato negro duerme]
- En BoW/TF-IDF con n-gramas:
  - Cada n-grama es una dimensión independiente
  - Se pueden combinar diferentes valores de n
  - Se suele usar  $n \leq 3$  por razones prácticas

- Los sistemas modernos de recuperación de información van más allá de la similitud vectorial:
  - PageRank
  - Relevance Feedback
  - Query log mining
  - Google Knowledge Graph
  - Machine Learning
- La recuperación de información y la minería de textos están menos preocupadas por la estructura lingüística y más interesadas en producir algoritmos rápidos y escalables [Eisenstein, 2018].

- Representar documentos como vectores sirve para calcular similitudes entre pares de documentos.
- Los vectores Bag of words y TF-IDF son dos representaciones que:
  - Carecen de estructura lingüística
  - Son de alta dimensión y dispersos
  - Pueden mejorarse con n-gramas para capturar expresiones multipalabra



Eisenstein, J. (2018).

**Natural language processing.**

Technical report, Georgia Tech.



Manning, C. D., Raghavan, P., and Schütze, H. (2008).

***Introduction to Information Retrieval.***

Cambridge University Press, New York, NY, USA.



Salton, G., Wong, A., and Yang, C.-S. (1975).

**A vector space model for automatic indexing.**

*Communications of the ACM*, 18(11):613–620.



Zipf, G. K. (1935).

***The Psychobiology of Language.***

Houghton-Mifflin, New York, NY, USA.